

Cloud Pipelines POC

Moving Delivery to Cloud-Based CI/CD

Presented by Jay Ntongwe



Azure DevOps · Azure Pipelines · Shift-Left Security

Agenda

1. Why change: the manual FTP status quo
2. The vision: cloud pipelines in Azure
3. Pipeline at a glance
4. Stage-by-stage tour: Build, SAST, MSDO, Deploy, DAST, Publish, DefectDojo
5. Security-first design: policy gates & reports
6. How it's built: a modular, reusable pipeline
7. Before vs. after
8. Benefits for the org
9. What's next: production hardening
10. Q&A

Where we are today

Shipping over FTP: manual, unguarded, unverifiable

The status quo: manual FTP delivery

Today's workflow

- Developer builds locally
- Files uploaded to the server by hand over FTP
- Copy-paste, drag-drop, "it works on my machine"
- Manual smoke test after upload
- No central record of what shipped or when

The problems

- No security checks: vulnerabilities ship silently
- No version traceability: what's on the server?
- Human error: wrong file, wrong server, partial upload
- No audit trail: who deployed what, and why?
- No central triage: findings live nowhere

[illegible]

The Vision

Move delivery into the cloud: automated, secure, and auditable on Azure

The vision: cloud-based pipelines

Automated end-to-end

Every push to `main`, `master` or `develop`: and every pull request against them: triggers the full pipeline automatically:

build → secure → deploy → test → report → (triage)

Security by design

- SAST: static analysis & secret scanning
- MSDO: Microsoft Security DevOps
- DAST: dynamic testing of the live app
- Policy gates: block critical & high findings
- Reports: always published, never lost

 Azure Pipelines + Azure DevOps = one repeatable, auditable release path

The pipeline at a glance

Pipeline flow

Always runs

Publish reports even when a scan fails: findings are never lost.

Gated

Critical/high findings fail the build before anything ships.

Optional

DefectDojo triage switches on via a single flag.

What runs where

#	Stage	Runs on	Purpose
1	Build	Hosted Linux	Publishes the app into the <code>drop</code> artifact
2	SAST	Hosted Linux	TruffleHog (secrets) + Semgrep (code) in parallel
3	MSDO	Hosted Windows	Microsoft Security DevOps → SARIF
4	Deploy	Self-hosted Windows VM	Installs the app as a Windows service
5	DAST	Self-hosted Windows VM	OWASP ZAP scans the running app
6	Publish	Hosted Linux	Aggregates findings → HTML dashboard, always runs
7	DefectDojo	<code>DefectDojoPool</code> agent	Uploads reports for central triage, optional

The main pipeline file is intentionally thin: it chains one template per stage. Most changes never touch pipeline logic.

Stage-by-stage tour

Build → SAST → MSDO → Deploy → DAST → Publish → (DefectDojo)

Build & SAST

1 Build

- Runs `dotnet restore` + `dotnet publish` on the sample app
- Verifies the output exists, then publishes the `drop` artifact
- A `copy` mode also ships pre-built/static content
- No security scanning here: that comes next

2 SAST: two jobs in parallel



TruffleHog scans the git history for leaked secrets & credentials



Semgrep performs static code analysis with rules

Each SAST job: install → scan → archive results → policy gate. If one scanner fails, the other still runs :output is always kept.

MSDO & Deploy

3 MSDO



Microsoft Security DevOps

- BinSkim, Bandit, Checkov, Terrascan, Trivy & more
- Runs on a hosted Windows agent
- Emits one consolidated SARIF report

4 Deploy

- Runs on the self-hosted Windows VM via agent pool `ccloud-poc`
- Stops the service → copies the artifact to `C:\site` → starts it
- App is hosted as a Windows service (survives between stages)
- Probes `http://localhost:5000` until it answers